usual `http://example.com/pt/books` versus `http://example.com/en/books`. Let's discuss the different options you have.

2.4 Setting the Locale from the Domain Name

One option you have is to set the locale from the domain name where your application runs. For example, we want `www.example.com` to load the English (or default) locale, and `www.example.es` to load the Spanish locale. Thus the *top-level domain name* is used for locale setting. This has several advantages:

- The locale is an *obvious* part of the URL.
- People intuitively grasp in which language the content will be displayed.
- It is very trivial to implement in Rails.
- Search engines seem to like that content in different languages lives at different, inter-linked domains.

You can implement it like this in your `ApplicationController`:

```
before_action :set_locale

def set_locale
  I18n.locale = extract_locale_from_tld || I18n.default_locale
end

# Get locale from top-level domain or return nil if such locale is not
available
# You have to put something like:
#   127.0.0.1 application.com
#   127.0.0.1 application.it
#   127.0.0.1 application.pl
# in your /etc/hosts file to try this out locally
def extract_locale_from_tld
  parsed_locale = request.host.split('.').last
  I18n.available_locales.map(&:to_s).include?(parsed_locale) ?
parsed_locale : nil
end
```

We can also set the locale from the *subdomain* in a very similar way:

```
# Get locale code from request subdomain (like
http://it.application.local:3000)
# You have to put something like:
#   127.0.0.1 gr.application.local
# in your /etc/hosts file to try this out locally
def extract_locale_from_subdomain
  parsed_locale = request.subdomains.first
  I18n.available_locales.map(&:to_s).include?(parsed_locale) ?
parsed_locale : nil
end
```

If your application includes a locale switching menu, you would then have something like this in it:

```
link_to("Deutsch",
"#{APP_CONFIG[:deutsch_website_url]}#{request.env['REQUEST_URI']}")
```

assuming you would set `APP_CONFIG[:deutsch_website_url]` to some value like `http://www.application.de`.

This solution has aforementioned advantages, however, you may not be able or may not want to provide different localizations ("language versions") on different domains. The most obvious solution would be to include locale code in the URL params (or request path).

2.5 Setting the Locale from the URL Params